

Fehlerbehandlungs- und Triage-Prozess für ORKA

Diese Seite beschreibt den Prozess der Fehlererfassung, bewertung sowie der Planung der Behebung von Fehlern.

Definition wesentlicher Begriffe

Die Definition wesentlicher Begriffe soll es erlauben einzelne Fehlerarten zu unterscheiden und mit einer klaren begrifflichen Definition arbeiten. Diese feiner Unterscheidung erlaubt es besser Fehler einschätzen und beheben zu können.

Quellen für die Begriffsdefintionen:

- ISO/IEC 24765:2017 - Systems and Software Engineering Vocabulary
- ISO/IEC 25010:2011 - System and Software Quality Models
- ISO/IEC/IEEE 12207:2017 - Software Lifecycle Processes

Begriff	Definition gemäß ISO/IEC	Quelle
Fehler (auch Bug, engl. Fault)	Eine Unvollständigkeit oder Mangelhaftigkeit im Code, der eine potenzielle Fehlfunktion verursacht.	ISO/IEC 24765:2017
Fehlverhalten, Ausfall, (engl. Failure)	Ein beobachtbares falsches Verhalten eines Systems oder einer Komponente während der Ausführung.	ISO/IEC 25010:2011
Irrtum, menschlicher Fehler (engl. Error)	Eine fehlerhafte Handlung oder Entscheidung, die zu einem Fehler führt.	ISO/IEC 24765:2017
Mangel (engl. Defect)	Ein nicht erfülltes Qualitätskriterium	ISO/IEC 25010:2011
Störung, Betriebsstörung, (engl Incident)	Ein unerwartetes Ereignis im Betrieb, das den normalen Ablauf beeinträchtigt.	ISO/IEC/IEEE 12207:2017

Der Vereinfachung halber arbeiten wir nur mit den Begriffen Fehler, Mangel, Störung. Die anderen Begriffe wurden nur der Vollständigkeit halber aufgeführt.

1. Eingang und Erfassung der Fehlermeldung

Jede Fehlermeldung von außerhalb des Teams ORKA wird über das zentrale Ticketsystem der DRV erfasst. Fehlermeldungen aus dem Team selber sollten ab einer gewissen Größe beim Arbeitsaufwand zur Behebung auch als Ticket Ticketsystem der DRV erfasst werden. Kann der Fehler mit sehr geringen Aufwand von ca. einer Stunde direkt behoben werden, muss dafür kein Ticket erstellt werden.

Jedes Ticket muss die folgenden Informationen enthalten:

- **Betroffene Umgebung** (Test, Produktion, ...)
- **Fehlerbeschreibung** (inkl. Screenshots, Logs, betroffene Module oder Services)
- **Reproduzierbarkeit** (Schritte zur Reproduktion, Häufigkeit)
- **Auswirkung auf die Nutzung** (Beeinträchtigung einzelner Funktionen oder des Gesamtsystems)
- **Betroffene Nutzeranzahl** (einzelne Nutzer, Gruppen, gesamte Produktion)

Fehlermeldungen sollten erst dann bewertet, eingeplant oder bearbeitet werden, wenn genügend Informationen hierfür vorliegen. Erlaubt die Auswirkung eines Fehlers, zum Beispiel eine Produktionsstörung, dies nicht, kann schon früher mit der Bearbeitung begonnen werden. Die fehlenden Informationen sind anschließend soweit wir möglich hinzuzufügen.

2. Triage-Prozess zur Fehlerbewertung und Priorisierung

Über das Ticketsystem erstellte Fehlermeldungen werden per E-Mail in regelmäßigen Abständen an das Triageteam des Teams ORKA gemeldet.

Zum Triage-Team gehören:

- Produkt Owner
- Architekt
- Lead Developer
- Scrum Master

Diese breite Aufstellung des Teams soll verhindern, dass dringende Fehler nicht erkannt und nicht bearbeitet werden.

Ein Mitglied des Teams übernimmt das Ticket und stuft es nach Rücksprache mit dafür geeigneten Personen (Entwickler, Melder, Produktionsteam) ein nach den Faktoren Schweregrad und Dringlichkeit.

Fehlermeldungen die eine sofortige Bearbeitung erfordern werden sofort nach Absprache mit Product Owner und Scrum Master in den aktiven Sprint eingearbeitet. Alle anderen Fehlermeldungen werden analysiert und in das Backlog des Teams eingefügt.

Sofern die Anzahl der gemeldeten, akzeptierten aber nicht gelösten Fehlermeldungen nicht mehr mit vertretbarem Aufwand im Rahmen der normalen Spring Planung besprechen und planen kann, wird hierfür ein Bugboard im Ticketsystem für das Team ORKA erstellt und in separaten Bug Product Backlog Refinement die einzelnen Fehlermeldungen besprochen und geplant. Dieses Vorgehen kann wieder beendet werden, wenn das Team als Ganzes die Meinung vertritt, dass die Menge der Fehler auf ein beherrschbares Mass abgesenkt worden ist.

2.1 Klassifikation nach Schweregrad (Severity)

Jede Fehlermeldung wird anhand ihrer **technischen und geschäftlichen Auswirkungen** bewertet.

Schweregrad	Kriterium	Beispiel
S1 – Kritisch	Produktionsausfall oder Sicherheitsrisiko	Login nicht möglich, Zahlungsfunktion defekt
S2 – Hoch	Hauptfunktionalität betroffen, aber System läuft	Verzögerung bei Transaktionen, API-Fehler
S3 – Mittel	Einzelnutzer betroffen oder Workaround verfügbar	UI-Fehler, sporadische Timeouts
S4 – Niedrig	Kosmetischer Fehler, Verbesserungsvorschlag	Rechtschreibfehler, nicht optimale UX

2.2 Klassifikation nach Dringlichkeit (Urgency)

- **U1 – Sofortige Reaktion:** Problem erfordert **sofortige Behebung** (z. B. Produktionsstopp)
- **U2 – Innerhalb eines Sprints:** Hohe Auswirkung, aber noch handhabbar
- **U3 – Geplant:** Kann in einer zukünftigen Version oder als Tech Debt behoben werden

2.3 Prioritätsmatrix (Severity vs. Urgency)

Die Kombination von **Schweregrad (S1–S4)** und **Dringlichkeit (U1–U3)** ergibt die finale **Priorität (P1–P4)**.

Schweregrad → / Dringlichkeit ↓	S1 (Kritisch)	S2 (Hoch)	S3 (Mittel)	S4 (Niedrig)
U1 – Sofort	P1 (Blocker, Hotfix nötig)	P1 (Schnelle Bearbeitung, innerhalb 24h)	P2 (Sprint)	P3 (Backlog)
U2 – Sprint	P1 (Innerhalb Sprint)	P2 (Hoch, Sprint einplanen)	P3 (Planung, nächste Iteration)	P4 (Gering, falls Zeit bleibt)
U3 – Später	P2 (Nächste Sprints, aber nicht sofort)	P3 (Backlog, Tech Debt)	P4 (Niedrig, Optional)	P4 (Nice-to-have)

3. Zuweisung & Bearbeitung

Nach der Triage erfolgt die **Zuweisung eines Entwicklers** oder eines Teams basierend auf der Priorität.